Quantum Simulator Challenge:

FX700 Environment Setup Guide for QARP

Fujitsu Quantum Research
Quantum Application Core Project

Introduction

Welcome to the Quantum Simulator Challenge on Fujitsu's HPC platform, FX700. This guide provides step-by-step instructions for setting up your environment on the FX700 platform to use Quantum Application Research Package (QARP).

This manual consists of the following parts:

- **Part 1 Standard Setup**:
  This section is for **all participants**. It covers the essential setup of the QARP and its standard dependencies. Completing this setup will enable you to run calculations using either QARP's default non-MPI Qulacs backend or the Qiskit backend. The Qiskit backend provides a Tensor Network simulator based on Matrix Product State (MPS).
- **Part 2 Optional Alternative Tensor Network Backend Setup**:
  This section is for participants who wish to explore another simulation technique using pytket-tenet backend. This backend is an interface to **Tenet.jl**, a library from BSC (Barcelona Supercomputing Center) written in Julia. It provides flexible functionalities for Tensor Network (TN) computations and is under active development with its capabilities continuously evolving. If you are not interested in this specific alternative TN-based method, you can skip this part.
- **Part 3 Verifying the Setup:**
  This section guides you through verifying your setup using sample code.

Note: This document focuses on the **environment setup on FX700**. For detailed information on the QARP's API and quantum chemistry concepts, please refer to the "**QARP Official User Manual**", which is available on the FX700 shared file system at:

- **/home/share/developer/QARPdemo/docs/QARP-doc**

Documentation for the optional pytket-tenet backend is available at:

- **/home/share/developer/QARPdemo/docs/pytket-tenet-doc**

These directories are accessible from compute nodes. If the path is not visible from the login node, please allocate a compute node (e.g., using `salloc`) before accessing the documentation.

For more detailed information on using the FX700, including how to allocate compute nodes and submit jobs, please refer to the "**Fujitsu HPC FX700 User Manual**".

Important Notice:

1. **Evaluation Criteria**:
   For the Quantum Simulator Challenge, **only the results obtained using QARP** will be considered for scoring. Participants may utilize alternative setups such as mpiQulacs for personal exploration. However, results generated using setups other than QARP will be excluded from the official scoring process.
2. **MPI Calculations with QARP and Qulacs:**
   By default, QARP uses the standard non-MPI Qulacs backend. QARP can optionally utilize Qulacs built with MPI support (MPI-enabled Qulacs) for limited use cases. Such usage is outside the scope of this manual and is documented separately. Fujitsu's proprietary "mpi-qulacs" library is not supported by QARP and cannot be used via the QARP framework.
3. **Confidentiality**:
   The QARP provided for this event is subject to the terms and conditions of the Simulator Trial Service Agreement between Fujitsu and the participant. In addition, QARP and any related materials constitute Fujitsu's Confidential Information as defined in the Fujitsu Quantum Simulator Challenge 2025-26 Event Participant Agreement. By using QARP, you agree to comply with these agreements.

Part 1: Standard Setup (Required for All Participants)

This section guides you through installing Python using pyenv, creating a dedicated environment, and installing the QARP library.

Step 1.1: Get the Sample Project

We have prepared a package containing sample code. This archive (**QARPdemo.tar**) is located on a shared file system. First, copy and extract it to your home directory.

```
[username@loginvm-XXX ~]$ salloc -N 1 -p Interactive --time=6:00:00
[username@fx-XX-XX-XX ~]$ cp /home/share/developer/QARPdemo/QARPdemo.tar ~/
[username@fx-XX-XX-XX ~]$ tar xvf QARPdemo.tar
```

Enter the newly created project directory. All subsequent commands should be run from within this directory.

```
[username@fx-XX-XX-XX ~]$ cd QARPdemo
```

Step 1.2: Install pyenv and Configure Environment

pyenv is a tool that allows you to easily install and manage multiple Python version.

1. Clone the pyenv repository from GitHub and install pyenv.

```
[username@fx-XX-XX-XX QARPdemo]$ git clone https://github.com/pyenv/pyenv.git ~/.pyenv
[username@fx-XX-XX-XX QARPdemo]$ cd ~/.pyenv && src/configure && make -C src
[username@fx-XX-XX-XX .pyenv]$ cd ~/QARPdemo
```

2. Configure your shell environment for pyenv.

```
[username@fx-XX-XX-XX QARPdemo]$ echo '
# pyenv configuration
export PYENV_ROOT="$HOME/.pyenv"
command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
' >> ~/.bashrc
```

3. Configure your shell environment for QARP. The following commands add the required Boost library path to your .bashrc file.

```
[username@fx-XX-XX-XX QARPdemo]$ echo 'export LD_LIBRARY_PATH=/home/share/developer/boost-1.90.0/lib:/usr/local/lib' >> ~/.bashrc
[username@fx-XX-XX-XX QARPdemo]$ echo 'export BOOST_ROOT=/home/share/developer/boost-1.90.0' >> ~/.bashrc
```

4. Reload your shell to apply the changes.

```
[username@fx-XX-XX-XX QARPdemo]$ source ~/.bashrc
```

Step 1.3: Install Python 3.12.10

Now, use pyenv to install Python 3.12.10.

1. Install Python 3.12.10. This step may take several minutes.

```
[username@fx-XX-XX-XX QARPdemo]$ pyenv install 3.12.10
```

2. Set the local Python version for your project directory.

```
[username@fx-XX-XX-XX QARPdemo]$ pyenv local 3.12.10
```

Step 1.4: Create and Activate a Virtual Environment

1. Create a virtual environment named venv.

```
[username@fx-XX-XX-XX QARPdemo]$ python -m venv venv
```

2. Activate it.

```
[username@fx-XX-XX-XX QARPdemo]$ source venv/bin/activate
```

Step 1.5: Install QARP (Manual Copy) and Dependencies for MPI-enabled Qulacs

The provided QARP package is distributed in a pre-compiled (.pyc) format. Therefore, it cannot be installed using standard pip install commands. Instead, you will directly copy the package into your virtual environment's site-packages directory.

1. Copy the qarp directory to your virtual environment's site-packages directory.

```
(venv) [username@fx-XX-XX-XX QARPdemo]$ cp -r qarp venv/lib/python3.12/site-packages/
```

2. Install dependencies for QARP from the provided requirements.txt file.

```
(venv) [username@fx-XX-XX-XX QARPdemo]$ pip install --no-cache-dir --extra-index-url https://pypi.org/simple -r requirements.txt
```

3. Exit environment.

```
(venv) [username@fx-XX-XX-XX QARPdemo]$ exit
```

```
[username@loginvm-XXX ~]$
```

Part 2: Optional Alternative TN Backend Setup

This section is for users who wish to use the advanced pytket-tenet backend. **If you are not interested in this specific alternative TN-based method, you can skip this part.**

The sample script, vqe.py, utilizes several libraries to run efficiently on the FX700 system:

- The script uses **mpi4py** to parallelize Pauli term expectation value calculations across multiple CPU nodes via MPI. You will install this package in this part.
- It also uses **scipy**'s classical optimizers (e.g., SLSQP) to find the ground state energy. scipy was already installed as a dependency of QARP in Part 1.

This part guides you through the complete setup process.

Step 2.1: Configure Environment

1. Configure your shell environment for pytket-tenet. Add the path to the GCC library to your .bashrc file.

```
[username@loginvm-XXX ~]$ echo 'export LD_LIBRARY_PATH=/home/share/developer/gcc-14.1.0/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
```

2. Reload your shell configuration.

```
[username@loginvm-XXX ~]$ source ~/.bashrc
```

Step 2.2: Install pytket-tenet

1. Allocate an interactive compute node and navigate to your project directory.

```
[username@loginvm-XXX ~]$ salloc -N 1 -p Interactive --time=6:00:00
[username@fx-XX-XX-XX ~]$ cd QARPdemo
```

2. Activate the virtual environment you created in Part 1.

```
[username@fx-XX-XX-XX QARPdemo]$ source venv/bin/activate
```

3. Move to pytket-tenet directory.

```
(venv) [username@fx-XX-XX-XX QARPdemo]$ cd pytket-tenet
```

4. Install pytket-tenet by running make install.

```
(venv) [username@fx-XX-XX-XX pytket-tenet]$ make install
```

Note: This installation process is expected to take a significant amount of time (potentially 10-20 minutes or more), as it involves compiling large components, including the Julia language runtime from source.

5. Apply MPI Compatibility Patch. This is a critical step to ensure pytket-tenet works correctly in a parallel MPI environment. A race condition can occur where multiple processes try to initialize the Julia environment simultaneously. We will apply a patch to serialize this initialization. The QARPdemo package includes the necessary patch file (juliacall__init__.diff).

```
(venv) [username@fx-XX-XX-XX pytket-tenet]$ patch ../venv/lib/python3.12/site-packages/juliacall/__init__.py juliacall__init__.diff
```

6. After the pytket-tenet installation is complete, navigate back to the project's root directory and install

mpi4py using pip.

```
(venv) [username@fx-XX-XX-XX pytket-tenet]$ cd ..
(venv) [username@fx-XX-XX-XX QARPdemo]$ pip install mpi4py
```

7. Exit environment.

```
(venv) [username@fx-XX-XX-XX QARPdemo]$ exit
[username@loginvm-XXX ~]$
```

Part 3: Verifying the Setup

Let's run example to confirm that the libraries are installed correctly.

Verifying the Base Installation (QARP only)

This check verifies that the core QARP is installed and functioning correctly on its own, using its default simulator. To run this verification, use the following steps in an interactive session.

1. Allocate an interactive compute node and navigate to the project directory.

```
[username@loginvm-XXX ~]$ salloc -N 1 -p Interactive --time=6:00:00
[username@fx-XX-XX-XX ~]$ cd QARPdemo
```

2. Activate your virtual environment.

```
[username@fx-XX-XX-XX QARPdemo]$ source venv/bin/activate
```

3. Navigate to the example directory and run mwe_vqe.py.

```
(venv) [username@fx-XX-XX-XX QARPdemo]$ cd example
(venv) [username@fx-XX-XX-XX example]$ python mwe_vqe.py
```

4. Expected Output: The calculation should converge and print the final energy of H_2 dimer, which should be very close to the E(CCSD) value shown in the output. A successful run will look similar to this:

```
converged SCF energy = -2.13221694379715
E(CCSD) = -2.202300469925308 E_corr = -0.07008352612815447
...
VQE Run:
Iteration        Energy              dE              |step|           |grad|
        1   -2.2022978452   -0.0000483991    0.0041514788    0.0034189511
        ...
        5   -2.2023004680   -0.0000000007    0.0000182214    0.0000046597
VQE minimization finished successfully
-2.202300469925308
```

Verifying with Tensor Network Backends

This verification is for users who have completed Part 2 and installed the optional pytket-tenet and qiskit-aer backends. The test will run by submitting a batch job. The sample project includes all necessary files, including the job script sim.job.

1. Make sure you are in the QARPdemo/tn_backend directory on a login node.

```
[username@loginvm-XXX ~]$ cd ~/QARPdemo/tn_backend
```

2. Submit the sample job using the sbatch command.

```
[username@loginvm-XXX tn_backend]$ sbatch sim.job
```

3. The job will be queued and executed on a compute node. You can check the job status using squeue.
4. Expected Output: The output of the job will be written to a file named test-<job id>. You should see the optimization results, indicating that the VQE calculation completed successfully. The output will look similar to this:

```
######################################################################
# VQE main
######################################################################
set_rnseed0 = 1
rnseed0 = 900
np.random.seed(900)
initial_theta_list=
[-1.78956080 -0.62043203 -1.17103488  0.14856492
  0.89365370 -1.57515461 -0.28851846 -0.15245460
 -0.77464191 -0.67697893  1.40744459]
     itr                  ctim          dtim          energy                  delta_e
       0 2025/12/23/20:56:18    100.3330      -7.751937919674242  0.000000000000000e+00
       1 2025/12/23/20:57:40     82.6515      -7.861251498271256 -1.093135785970141e-01
       2 2025/12/23/20:58:56     75.9102      -7.861075702033482  1.757962377739730e-04
       3 2025/12/23/21:00:12     76.1204      -7.861375721046192 -3.000190127098179e-04
[info]
solution status : True
iteration           : 3
minimum value     : -7.861375721046192
minimizer           : [-1.78956061 -0.62043214 -1.17103476  0.14856492
  0.89365370 -1.57515461 -0.28851863 -0.49999913
 -0.77464191 -0.67697893  1.40744459]
function calls   : 37
elapsed time(s) : 234.6820
# of params        : 11
# of node           : 2
```

Important Note on First-Time Execution and Job Submission

- **First-Time Execution**: The first time you run a job using the pytket-tenet backend, the execution may take significantly longer than subsequent runs. This is because Julia's Just-In-Time (JIT) compiler is performing **a one-time "precompilation"** of the underlying libraries (like PythonCall.jl). This is expected behavior and could potentially take up to 2 hours or more for the first run. If this initial setup takes an excessively long time, please contact the support team.
- **Submitting Jobs**: Due to the nature of Julia's environment initialization, please submit jobs that use the

pytket-tenet backend **one at a time**. Submitting multiple jobs simultaneously can cause file-access conflicts.

If your job completes successfully and produces the expected output, your environment is correctly configured to use the optional TN backend. For details on the code and input files used in this sample job, please refer to the Appendix.

Appendix A: Details of the TN Backend Sample Job

This appendix provides details on the files used in the TN backend verification job (sim.job). All these files are included in the QARPdemo.tar archive.

A.1 Job Script (sim.job)

This is an example of a batch job script for the FX700 system using Slurm. It sets up the environment and runs the MPI-based Python script.

A.2 Main Python Script (vqe.py)

This is the main script that performs the VQE calculation. It handles MPI communication, circuit construction, and energy minimization.

A.2.1 How QARP Leverages Different Backends

A key feature of QARP is its ability to direct expectation value calculations to different simulation backends via the qarp.engines.TketEngine. By changing the backend object passed to the TketEngine, you can easily switch the underlying simulation method.

In the provided vqe.py, the pytket-tenet backend (specifically MPSInnerProductBackend) is configured within the sp_cost0 function, as shown below:

```
# --- Example: How to switch to the Tenet backend ---
from pytket.extensions.tenet import MPSInnerProductBackend, Config
# Engine
config = Config(mps_bond_dim=bond_dimension, transpile_to_mps=False)
TenetBackend = MPSInnerProductBackend(config)
engine = TketEngine(backend=TenetBackend)
```

If you wanted to switch to the qiskit-aer simulator, you would simply change this part of the code as follows.

```
# --- Example: How to switch to the qiskit-aer backend ---
from pytket.extensions.qiskit import AerBackend
# Engine
AerBackend = AerBackend()
engine = TketEngine(backend=AerBackend)
```

A.3 Input Parameter File (vqe_in.py)

This file defines all the input parameters for the vqe.py script. Users can modify this file to change the molecule, ansatz, or optimizer settings.

Explanation of Key Input Parameters in vqe_in.py:

- ham_read: If True, the script reads a pre-calculated Hamiltonian from the file specified by ham0_ifile.

If False, it calculates the Hamiltonian using PySCF.

- actidx: A list of active orbital indices used to define the active space for the calculation. The number of qubits will be 2 * len(actidx).
- n_elec: The number of electrons in the active space.
- n_depth: The depth of the quantum circuit ansatz.
- qcansatz: The type of ansatz to use. Options include "Hardware", "SymPrev", and "SymPrevReal".
- sp_method: The optimization method for SciPy's minimize function (e.g., 'SLSQP', 'BFGS').
- sp_options: A dictionary of options for the selected optimizer, such as the maximum number of iterations ("maxiter").
- bond_dimension: This parameter controls the bond dimension of the Matrix Product State used by the TN backend. A larger bond dimension generally leads to higher accuracy but also increases computational costs (default 2).

A.4 Ansatz Definition File (ansatz.py)

This file contains the Python classes that define the structure of the different quantum circuit ansatz available in the sample code.